



Traffic Shaping (Part 2)

Purpose of this lab:

This lab uses the token bucket implementation (from Lab 2a) for experiments with traffic shaping. The traffic for testing the token bucket will be the Poisson, VBR video, and LAN traffic traces from Lab 1.

Software Tools:

- This lab will use the software code from Lab 2a.

What to turn in:

- On Crowdmark: A report with your answers to the questions in this lab, including the plots.
- On Quercus: Your modified code files.

February 2026

© Jörg Liebeherr, 2007-2026. All rights reserved. Permission to use all or portions of this material for educational purposes is granted, as long as use of this material is acknowledged in all derivative works.

Table of Content

COMMENTS	2
PART 1. A REFERENCE TRAFFIC GENERATOR	3
<i>Exercise 1-a. Create and Test Reference Traffic Generator.....</i>	3
PART 2. TOKEN BUCKET IMPLEMENTATION.....	4
<i>Exercise 2-a. Complete the implementation of the token bucket</i>	4
<i>Exercise 2-b. Validate your implementation</i>	4
<i>Exercise 2-c. Maximum rate of token bucket</i>	6
PART 3. ENGINEERING TOKEN BUCKET PARAMETERS.....	7
<i>Exercise 3-a. Assume that (long-term) bandwidth is cheap.....</i>	7
PART 4. (OPTIONAL) SHAPING VBR VIDEO WITH MULTIPLE TOKEN BUCKETS.....	9
<i>Exercise 4-a. Dual Token Bucket Traffic Shaper</i>	9
<i>Exercise 4-b. Selecting dual token bucket parameters for a VBR video source</i>	10
FEEDBACK FORM FOR LAB 2B	11

Comments

- Part 4 is optional for an extra credit of (max.) 20%.

Part 1. A Reference Traffic Generator

In this part of the lab, you create a simple traffic generator that can be used for testing and tuning your token bucket implementation from Lab 2a. This reference traffic generator creates a predictable periodic traffic pattern of UDP datagrams. The traffic generator takes as input parameters three values T , N , L , with the interpretation:

“Every T milliseconds transmit a group of N packets back-to-back, each with a size of L bytes”

If the group of N packets are sent quickly, then the traffic generator sends traffic according to an arrival function A with:

$$\begin{aligned} A(t) &= \lceil t/T \rceil \cdot N \cdot L \text{ Bytes} \\ &= \lceil t/T \rceil \cdot N \cdot L \cdot 8 \text{ Bits} \end{aligned}$$

The long-term average traffic rate of the traffic generator is given by $Rate_{source} = NL8/T$ bps (bits per second).

For each transmitted packet, write a line to an output file that records the size of the packet and the time since the transmission of the previous packet (For the first packet, the time is zero), e.g., the file has the format:

SeqNo	Time since last arrival (in msec)	Size (in Bytes)
1	0	320
2	934	99
3	2293	27

Exercise 1-a. Create and Test Reference Traffic Generator

Step 1: Build a traffic generator for the period traffic pattern described above by modifying the implementation of the traffic generator from Lab 2a (Part 2).

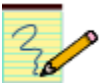
Step 2: Run experiments for specified values of T , N , and L , where you transmit traffic from the traffic generator to the traffic sink (without a token bucket). Evaluate the accuracy of the traffic generator by comparing the entries in the trace file (at the traffic generator) to the results written to the output file (at the sink). For each experiment, only provide 500 data points.

- $N = 1$, $L = 1250$ byte, $T = 1$ ms
- $N = 10$, $L = 1250$ byte, $T = 10$ ms
- $N = 4$, $L = 625$ byte, $T = 20$ ms

Note that all three scenarios create traffic at a rate of 10 Mbps

Step 3: Prepare a plot that shows the difference of trace file and the output file. For example, you may create two functions that show the cumulative arrivals of the trace file and the output file, respectively, and plot them as a function of time.

Lab Report



- Provide the generated plots with a brief description. Upload the software code to Quercus.

Part 2. Token Bucket Implementation

In this part you implement the missing features from the token bucket implementation provided to you for Lab 2a.

You only need to modify the implementation of the *TokenBucket* class. The provided implementation does not correctly update the value of TB and does not correctly compute the time for waking up *TokenBucketSender*. An instance of the *TokenBucket* is created by calling the constructor of this class with two parameters *size* and *rate*, where

- *capacity* is the maximum content of the token bucket in tokens (TB_{max}),
- *rate* is the token generation rate in tokens per second (r).

Other classes can query the *TokenBucket* about the content of the token bucket, and can request to remove tokens from the bucket. The following requests are available:

- *getNoTokens()*: Returns number of tokens in bucket.
- *removeTokens(X)*: Request to remove X tokens from the bucket. The method returns false if there are less than X tokens in the bucket.
- *getWaitingTime(X)*: Returns the waiting time (in milliseconds) until bucket has X tokens. Note that there is no guarantee that there will be enough tokens at the returned time (someone else may have removed tokens).

Exercise 2-a. Complete the implementation of the token bucket

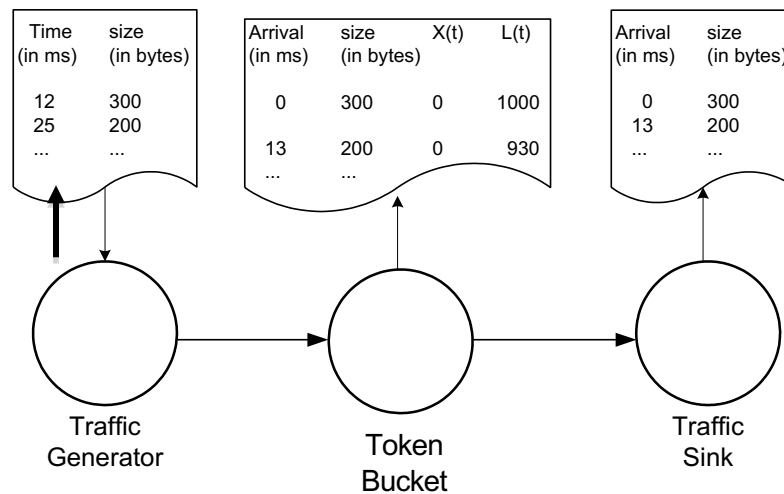
Step 1: Complete the implementation of the *TokenBucket* class by providing the algorithms for updating the contents of the token bucket, and executing the requests *getNoTokens*, *removeTokens*, *getWaitingTime*.

Hints:

- The contents of the token bucket does not need to be kept updated at all times. It is sufficient to update the token bucket only when a request has been made to *TokenBucket*.
- Be consistent about whether a single token corresponds to a Bit or a Byte.

Exercise 2-b. Measurement experiments

Design and conduct a series of measurement experiments that show that your implementation of the token bucket is correct. The setup of the experiments is as seen in Lab 2a:



Use the reference traffic generator from Part 1 to create a deterministic traffic stream with known properties.

When you conduct a measurement experiment, record relevant data (The code for this should already be available):

- **At the traffic generator:** For each transmitted packet, record the size of the packet and the time since the previous packet transmission (For the first packet, the time is zero).
- **At the token bucket:** Upon each packet arrival, write a line to an output file that records the size of the packet and the time since the arrival of the last packet (For the first packet, the time is zero). Also recorded are the number of tokens ($TB(t)$) in the token bucket and the backlog in the buffer ($B(t)$) after the arrival of the packet.
- **At the traffic sink:** For each incoming packet, write a line to an output file that records the size of the packet and the time since the arrival of the previous packet (For the first packet, the time is zero).

For the parameters of the traffic generator (L , N , T), use the three scenarios in Exercise 1-b. All three scenarios generate traffic at an average rate of 10 Mbps.

Step 1: Conduct the measurement experiments of the three traffic scenarios from Exercise 1-b on a token bucket with the following parameters:

rate = 10 Mbps, size = 1250 Bytes.

Make sure you record the timestamped data at traffic generator, token bucket, and traffic sink.

Step 2: Use the data saved in the measurement experiments to generate a series of plots. For each measurement experiment above, prepare a single plot that shows the cumulative arrival function as a function of time of:

- The transmissions of the reference traffic generator;

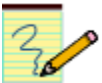
- The arrivals at the token bucket;
- The arrivals at the traffic sink.

For each experiment, provide a second plot that shows the content of the token bucket and the backlog in the Buffer as a function of time.

Exercise 2-c. Maximum rate of token bucket

Step 1: Determine the maximum arrival rate of traffic that can be supported by your token bucket. Support your findings with an experiment and (at least) one data plot.

Lab Report



- Describe the design of your implementation for the token buffer. Upload the source code to Quercus. (Only include source code of methods that you have modified).
- For Exercises 2-b, provide the set of plots and include a brief description of the plots. Describe your observations in each of the experiments.
- For Exercise 2-c describe how you determined the maximum supported arrival rate and discuss any graph that you generated to support your findings.

Part 3. Engineering Token Bucket Parameters

In this part you determine suitable token bucket parameters for two traffic sources used in Labs 1 and 2a: compressed video (*movietrace.data*) and the Bellcore Ethernet traffic trace (*C-pAug89.TL.Z*). For each tracefile, only consider the first 1000 entries.

The token bucket parameters must be selected subject to meet (all) of the following constraints:

- The objective is to smooth the traffic as much as possible, by spacing out packets, but without delaying traffic by more than 100ms (milliseconds).
- The required size of the buffer in the token bucket should be as small as possible, but the buffer should be large enough so that no overflows occur.
- The rate of the token bucket should be at least equal to the average rate of the traffic source, and less than the peak rate of the traffic source.

There is generally a whole family of solutions for the token bucket parameters (*size*, *rate*) that satisfy the above constraints. Your choice of parameters must trade-off the desire to smooth the traffic rate (small value for *rate*) or to reduce the burst size of the output of the token bucket (small value for *size*)

Note: Using the topics covered in the lecture, you can compute the required parameters without resorting to trial and error. To do this, you need to compute the empirical envelope of each traffic source, and use the service curve $S(t) = \text{size_TB} + \text{rate_TB} * t$.

Exercise 3-a. Assume that (long-term) bandwidth is cheap

Step 1: Determine the average and peak rate of the selected 1000 entries for the compressed video and Bellcore trace.



Computing the peak rate

The peak rate of a traffic source is the maximum rate over a short interval. Given a tracefile with timestamps and arrival times (of packets or video frames), proceed as follows:

1. Ignore the first arrival in the tracefile.
2. For all other arrivals, compute the ratio of the size of the arrival and the elapsed time since the previous arrival
3. The largest ratio is the peak rate.

Step 2: For each of the three traffic sources (Video, Ethernet), determine token bucket parameters that satisfy the above constraints. When you have to make a choice between increasing the rate parameter or the size parameter, your preference should be to increase the rate parameter.

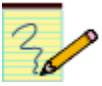
Step 3: For the parameters that you select, generate a plot for the cumulative arrival function for arrivals at the token bucket and arrivals at the traffic sink.

Note: Transmitted packets are not permitted to exceed 1480 Bytes. Therefore, video frames must often be divided up into multiple packets.

Step 4: Also plot the content of the token bucket and the backlog in the buffer as a function of time.

Exercise 3-b. Assume that (long-term) bandwidth is expensive

Step 1: Repeat the steps of Exercise 3-a, but with a different assumption. Now, when you have to choose between increasing the rate parameter or the size parameter, your preference is to increase the size parameter.



Lab Report

- Provide the average and peak rates computed in Exercise 3-a.
- Discuss your methodology for selecting the token bucket parameters in Exercises 3-a and 3-b.
- For Exercises 3-a and 3-b, include the set of plots with a brief description. Describe your observations in each of the experiments. Compare the results for the cheap bandwidth and expensive bandwidth scenarios.
- Compare the results for the three traffic types. For example, which type of traffic is most demanding in terms of bandwidth or memory requirements? Which type of traffic benefits the most from traffic shaping? Support your findings with data (plots).

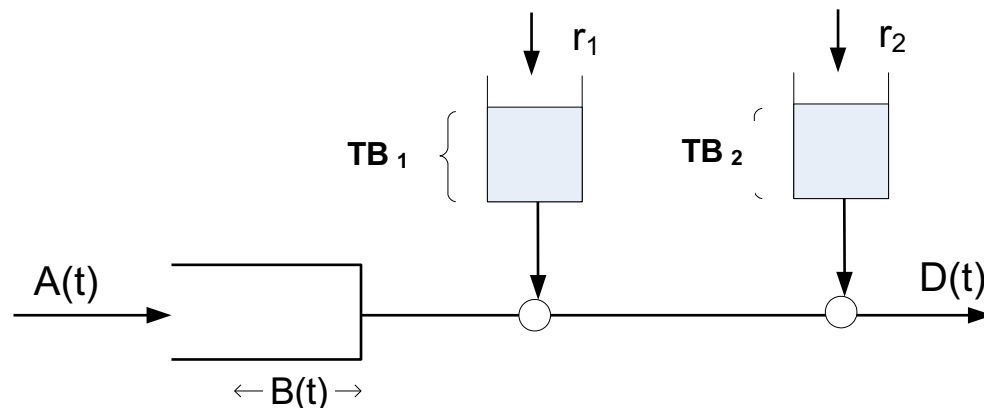
Part 4. (Optional) Shaping VBR Video with Multiple Token Buckets

The regulation of VBR video traffic with a token bucket has to address the following two problems:

- Since the size of a frame can be much larger than the maximum packet size (in the video trace file, the maximum frame size exceeds 600 KB), yet the time lag between frames is relatively long, one would like to select token bucket parameters that stretch the transmission of a frame over the entire 33 ms.
- In order to reduce the long-term resource consumption of the VBR video source, one would like to set the token bucket rate parameter to a value that is close to the average rate of the VBR source.

Both constraints, however, cannot be satisfied by a (simple) token bucket. The solution is to regulate VBR video traffic by two token buckets put in series:

- The first token bucket controls the peak rate, by spacing out the transmission of a frame over a longer duration;
- The second token bucket controls the average rate of the VBR source.
- An illustration of a dual token bucket is shown in the figure below. An arriving packet must withdraw tokens from both token buckets. If one of the two buckets does not have enough tokens, the packet must wait until sufficient tokens are available in both buckets.



Exercise 4-a. Dual Token Bucket Traffic Shaper

Step 1: Revise the token bucket implementation to build a traffic shaper that realizes a dual token bucket. The token bucket has four parameters (TB_1 , r_1 , TB_2 , r_2), for the size and rate of the token buckets. As before, the Token Bucket receives data from a traffic generator and transmits the output to a traffic sink.

Step 2: Build a traffic generator (of your choice) that can be used to demonstrate the correctness and accuracy of your implementation.

Step 3: Provide plots that illustrate how you tested the implementation.

Exercise 4-b. Selecting dual token bucket parameters for a VBR video source

Your task is to perform traffic shaping of a VBR video trace with your dual-token bucket implementation.

Step 1: Provide a traffic generator that transmits video using the tracefile *movietrace.data*.

- Transmitted packets are not permitted to exceed 1480 Bytes. So, some frames must be divided up into multiple packets.
- As before, only consider 1000 entries from the tracefile.

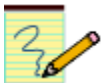
Step 2: Make suitable parameter selections for the dual token bucket from Exercise 4-a. The selection of parameters should reflect a tradeoff between multiple considerations:

1. The maximum buffer size (and the maximum waiting time) should not be too large;
2. The average rate allocation should not be much larger than the average rate of the VBR source (to avoid over-allocation of bandwidth);
3. The number of packets that can be transmitted at once (this is determined by $\min(TB_1, TB_2)$), should be small.

Step 3: Prepare a plot that shows the differences between the trace file and the output file, as a function of time. Prepare plots that depict the content of the token bucket and the backlog in the buffer as functions of time.

- What percentage of time is there a backlog at the token buckets?
- What is the longest backlog at the token buckets?

Lab Report



- Describe the design of your implementation for the dual token bucket. Include your source code in the lab report. Include and discuss how you have tested the implementation.
- Discuss your method for selecting the token bucket parameters. Use plots to justify your choices.
- Provide the plots and the answers from Exercise 4-b.

Feedback Form for Lab 2b

The lab has been revised for the W2026 semester. We appreciate any feedback on any issues that you may have encountered, e.g.,

- Instructions that are in need of clarification;
- Issues with provided Python code;
- ...

If you have feedback, please add it to your lab report as an appendix.

Thank you!